The University of Hong Kong

School of Computing and Data Science

COMP7705

Project Report

Semantic-Motion: A Multi-view Engineering Pipeline for Perception, Fact-preserving Language Augmentation and Fail-closed Multimodal Refinement

Submitted in partial fulfillment of the requirements for the admission to the degree of Master of Science in Computer Science

By

Gai Yifan(), Yang Tongzhou(), Li Chenyang (3036657835)

Mentor: Prof. Taku Komura

Date of submission: 17/07/2026

Declaration

State originality, academic integrity, use of generative AI where required, and individual contributions for a group project.

Abstract

Keep the abstract below 200 words. State the data problem, the four-module pipeline, the evaluated outputs, the strongest quantitative result, and the main limitation. Do not describe planned work as completed work.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Context: From Motion Data to VLA-ready Multimodal Data

Recent vision-language-action (VLA) systems seek to map visual observations and natural-language instructions directly to robot actions. Representative systems such as RT-2 encode actions as tokens and co-train on web-scale vision-language data and robot trajectories [11], while OpenVLA provides an open 7-billion-parameter implementation trained on a large and diverse collection of robot demonstrations [3]. These developments make the availability of aligned multimodal training examples increasingly important. A useful demonstration is no longer only a video or a joint trajectory; it is a traceable tuple linking what was observed, how the body or robot moved, what task was performed, and which temporal interval supports the description.

The motivating data source for this project is loco-manipulation motion: whole-body movement in which locomotion, posture change, reaching and interaction may occur in the same sequence. The Master Proposal identifies a representation gap between such motion data and the inputs expected by vision-language systems. Raw FBX/BVH animation and frame-wise 3D joint coordinates are useful for kinematic analysis, but a general-purpose VLM does not directly consume these representations. Conversely, video alone hides the precise trajectory information required to assess smoothness, acceleration, jerk and temporal coverage. The project therefore treats video, motion and language as complementary views of one sample rather than as interchangeable data products.

This motivation is consistent with the broader move towards heterogeneous robot data. Open X-Embodiment aggregates data from many institutions and embodiments to study cross-robot transfer [8], while Being-H0.5 explicitly investigates human-centric pre-training and cross-embodiment generalisation [5]. Semantic-Motion does not attempt to reproduce those large policy-training systems. It addresses an earlier engineering question: how can raw or rendered demonstrations be converted into auditable *candidate* Video–Motion–Text records before they are admitted to a VLA training corpus?

The implemented system answers this question through four modules. In operational order, Module D renders motion into synchronized fixed and egocentric videos and exports 3D trajectories; Module A extracts temporally grounded macro-intents and micro-instructions; Module B rewrites those instructions into language variants; and Module C reconstructs standardized samples and evaluates physical motion quality and text–video consistency. The modules are joined by explicit JSON/JSONL interfaces and are exposed through both command-line entry points and a Web-based orchestration dashboard.

1.2 Engineering Problem and Motivation

The central engineering problem is not simply video captioning. It is the construction of a repeatable data transformation in which evidence remains aligned across camera views, time, motion and generated language. Five coupled failure sources make this difficult.

1. **Representation mismatch.** Motion-capture files and simulation trajectories are not directly

interpretable by a VLM, whereas a rendered video does not preserve exact physical derivatives.

2. **Temporal ambiguity.** A long demonstration can contain several tasks and transitions. Uniformly assigning one caption to the whole video can merge unrelated actions, while isolated frame captions lose action order and task boundaries.
3. **View-dependent evidence.** A fixed camera is effective for whole-body displacement and scene geometry, but an egocentric view can better expose local interaction. Treating either view as complete can produce unsupported semantic claims.
4. **Generative uncertainty.** VLM annotations and LLM rewrites can omit actions, invent objects, reverse temporal order, or add plausible but unobserved details.
5. **Silent data-quality failure.** A linguistically plausible description may be paired with a noisy or incomplete trajectory. Conversely, a smooth trajectory does not guarantee that the generated text describes the corresponding video.

Large demonstration datasets illustrate why scalable processing matters. BridgeData V2 collects diverse manipulation trajectories across multiple environments [10]; MimicGen shows how a small number of human demonstrations can be transformed into much larger simulated datasets [6]. At such scales, manual inspection of every Video–Motion–Text tuple is impractical. The required solution is therefore a modular and observable pipeline that rejects malformed interfaces deterministically, attaches machine-readable reasons to suspicious samples, and retains enough provenance for later human audit.

1.3 Project Aim and Engineering Deliverables

The project aim is to design and implement an end-to-end engineering pipeline that converts paired visual and physical motion evidence into structured, language-annotated demonstration samples and evaluates whether those samples are suitable candidates for downstream VLA research.

The intended deliverables are:

- a Blender-based rendering workflow for FBX/BVH motion that produces synchronized `Fixed_View.mp4` and `Ego_View.mp4` streams together with a real 3D trajectory;
- a manifest generator that assigns a stable `sample_id` and joins each pair of views to its trajectory;
- a paired-view semantic perception pipeline with overlapping macro windows, weighted transition aggregation, dense micro windows and structured task records;
- a language augmentation interface supporting source-only, deterministic template and API-backed LLM rewriting modes, with provenance and audit metadata;
- a Module C preparation and refinement workflow that normalizes motion tracks, verifies cross-modal alignment, computes motion-quality indicators, performs multi-view semantic verification and emits decisions with stable reason codes;
- independent per-sample JSON and JSONL artifacts, avoiding hidden concatenation or filename collisions;
- a FastAPI and React/Vite control surface for asynchronous execution, progress polling, captured subprocess logs and visualization of Motion Quality, Semantic Consistency, Keep/Discard decisions and reason codes; and
- tests and explicit metadata sufficient to distinguish automatically verified behavior from API-dependent, Blender-dependent and human-annotation-dependent evidence.

1.4 System Requirements

The requirements below are phrased as engineering acceptance criteria. Where the present implementation only partially meets a criterion, the limitation is stated explicitly rather than treating planned work as completed.

Functional requirements

FR1: Rendered evidence For each valid source motion, the rendering stage shall produce fixed and egocentric videos plus a trajectory containing frame-wise positions for configured tracks such as Root, Hand_R and Hand_L.

FR2: Manifest indexing Each sample shall have a unique identifier, two resolvable view paths and a resolvable trajectory path. Invalid directories shall be reported rather than silently indexed.

FR3: Shared temporal contract All views shall be represented against one FPS, frame count, duration and frame map. Perception shall sample identical source-frame indices from selected views.

FR4: Multi-granularity perception The system shall represent high-level task intent, temporally ordered micro-instructions, objects, spatial relations, body part, contact state, posture, confidence and evidence-frame references.

FR5: Controlled augmentation Every generated instruction variant shall retain its source-step references, rewriter identity, model and prompt provenance. The design objective is fact-preserving augmentation. The current API-backed rewriter relies on prompt constraints and records `validation=disabled`; a deterministic post-generation fact gate remains an acceptance gap.

FR6: Multimodal refinement Module C shall load the real trajectory, normalize supported spatial units, check paired-view/time/motion coverage, score motion tracks, and independently judge text–video consistency.

FR7: Explainable output Refinement shall emit `motion_quality_score`, `semantic_label`, `semantic_confidence`, `decision`, `reason_codes` and diagnostic auxiliary data.

FR8: Independent artifacts Perception, prepared-sample and refined outputs shall be written under sample-specific filenames.

FR9: Selective and batch execution The orchestrator shall process all manifest entries by default and optionally restrict execution to a validated `target_sample_id`.

FR10: Observability An operator shall be able to start a task, poll its state, inspect stage progress and logs, and retrieve available refined results without blocking the browser request for the duration of model inference.

Non-functional requirements

NFR1: Traceability Outputs shall preserve sample identifiers, source paths, dataset identity, generator, Git revision, evidence intervals and configuration metadata where available.

NFR2: Modularity Rendering, perception, augmentation and refinement shall remain callable as separate stages with file-based contracts.

NFR3: Robust failure reporting Missing files, unsupported units, unreadable videos, subprocess failures and API failures shall be surfaced through exceptions, task status or reason codes.

NFR4: Reproducibility Deterministic windowing and scoring shall be configuration-driven. Ex-

ternal VLM/LLM responses must be marked as model- and API-dependent.

NFR5: Resource accessibility The semantic stages shall be runnable without local model training by using an OpenAI-compatible cloud endpoint, while acknowledging the resulting cost and reproducibility trade-offs.

1.5 Scope and Boundaries

Semantic-Motion is a data engineering and evaluation system, not a robot controller. It does not train an action policy, execute actions on hardware, or claim that accepted samples improve downstream task success. The VLA-ready designation means that records expose aligned modalities and auditable metadata; it is not a certification of compatibility with every policy architecture.

The formal paired-view path requires real fixed/ego videos and a real motion trajectory. Single-video, source-only, template-rewriter, dummy-motion and mock semantic-verifier modes are useful for debugging, but they do not constitute formal multimodal evidence. Module D depends on Blender and an appropriately configured scene, while semantic generation and verification depend on an external Qwen-compatible endpoint. These dependencies are outside the repository's deterministic test boundary.

The current evaluation assets are small and partly synthetic. LIBERO task names are weak task-level labels, not human gold labels for temporal boundaries, contact states or keep/drop decisions. Human annotation packets must be independently completed and adjudicated before formal precision, recall or calibration claims are made. In addition, although the project objective is fail-closed refinement, the current binary decision in `src/module_c/refinement.py` drops a sample when the motion score is below threshold or the semantic label is not `consistent`. Synchrony, semantic-confidence and mock-verifier findings are recorded in `reason_codes`, but not all currently alter the binary decision. This is treated as an explicit implementation gap.

1.6 Contributions

The principal contribution is a runnable and inspectable interface between motion rendering, video semantics, language generation and quality control. More specifically, the project contributes:

1. a versioned `ViewBundle` contract that links synchronized camera streams, a shared clock, a motion reference and provenance;
2. a paired-view temporal perception implementation that interleaves fixed/ego evidence at common timestamps, combines overlapping macro-window transition votes using Hanning weights, and derives dense micro-level descriptions;
3. a common augmentation interface with explicit source-only, template and LLM-backed modes and auditable rewrite metadata;
4. a standardized Module C schema and multi-track motion-quality calculation that exposes velocity, acceleration, jerk, jitter and temporal regularity diagnostics;
5. joint multi-view semantic verification and machine-readable explanations for refinement outcomes;
6. a per-sample automation layer that validates manifest paths, preserves independent artifacts and records subprocess output; and
7. a Web dashboard that exposes orchestration state and final evaluation dimensions without replac-

ing the underlying command-line interfaces.

These contributions are primarily architectural and engineering contributions. The report does not claim a new foundation model, a new policy-learning algorithm, or statistically established superiority over large-scale VLA systems.

1.7 Report Organisation

Chapter 2 reviews VLA systems, embodied VLMs, robot-learning benchmarks, video-to-task generation and demonstration-quality systems. Chapter 3 derives the system architecture and interface contracts from the requirements. Chapter 4 describes the implementation of the rendering, perception, augmentation and refinement modules. Chapter 5 reports integration and evaluation evidence. Chapter 6 discusses strengths, failure modes, limitations and engineering trade-offs, and Chapter 7 concludes with future work.

Chapter 2

Technical Background and Related Systems

2.1 Vision-Language-Action Models

VLA models extend vision-language reasoning to action generation by conditioning a policy on observations and language. RT-2 demonstrates one influential approach: robot actions are discretized into text tokens so that a vision-language model can be co-fine-tuned on robot trajectories and web data [11]. OpenVLA instead emphasizes accessibility, releasing a 7B-parameter VLA and training infrastructure based on a large collection of real robot demonstrations [3]. Open X-Embodiment further shows the scale and heterogeneity of data required to study transfer across institutions and robot embodiments [8].

These systems motivate Semantic-Motion in two ways. First, language-conditioned control assumes that instructions are grounded in the corresponding visual and physical sequence. Second, pooling data across sources increases the importance of explicit identifiers, coordinate conventions, timebases and provenance. Semantic-Motion therefore operates upstream of policy learning: it creates and checks candidate multimodal records but does not generate control actions.

Being-H0.5 is particularly relevant to the proposal because it frames human-centric data as a resource for cross-embodiment transfer [5]. The present project adopts the motivation, but not the claimed scale or policy architecture. Human skeletal trajectories rendered in Blender remain separated from robot action spaces, and the embodiment gap is a core limitation rather than an assumed solved problem.

2.2 Robot Learning Benchmarks

LIBERO provides 130 procedurally generated manipulation tasks organized into four suites and is designed to study declarative and procedural knowledge transfer in lifelong robot learning [4]. RoboCasa supplies a large-scale household simulation platform with diverse tasks, scenes and objects [7]. Both are useful references for task categories and structured demonstrations, but neither directly supplies gold labels for the temporal micro-actions and semantic consistency decisions defined by this project.

Semantic-Motion uses benchmark resources at two levels. A small synthetic image benchmark supports static tests of object recognition, task classification, action sequence, semantic similarity and spatial reasoning. Separately, selected LIBERO Goal demonstrations can be converted into paired agent-view and eye-in-hand videos with end-effector trajectories. The latter are more representative of the system's contracts, but the BDDL task title is only a weak video-level label. It cannot validate contact state, precise temporal boundaries or refinement decisions without additional human annotation.

Open X-Embodiment [8] and BridgeData V2 [10] provide a broader data-engineering context.

Their diversity demonstrates the value of normalized interfaces, but the current project does not claim direct ingestion of all their native formats. Its implemented adapters cover Module D trajectories, selected LIBERO exports and standard single- or multi-track motion JSON.

2.3 Vision-Language Models for Robotics

VLMs provide a practical interface between visual evidence and structured language. PaLM-E interleaves continuous sensor representations and text within an embodied language model and demonstrates transfer across embodied reasoning, visual question answering and planning tasks [2]. RT-2 uses vision-language pre-training more directly for action prediction [11]. Semantic-Motion uses the same general grounding capability for a narrower purpose: observation and verification.

In Module A, a Qwen-compatible VLM receives sampled frames and is prompted to return objects, spatial relations, task type, action order, transition indices, body part, posture and contact information. In Module C, an independently configured verifier receives the candidate text together with one or both views and returns a normalized label, confidence and error types. This separation is important: generation confidence is not treated as proof that a description is faithful.

However, an API response is neither deterministic nor self-validating. Model updates, sampling, image compression, prompt changes and service failures can alter outputs. The system consequently stores raw responses and model/prompt metadata where possible and distinguishes deterministic schema checks from model-mediated judgments.

2.4 Video-to-Task Generation

Long demonstration videos create a joint segmentation and labelling problem. Video2Tasks is an open-source system that processes overlapping windows, uses a VLM to propose task transitions and instructions, and aggregates transition evidence with Hanning weights [9]. It is a software baseline rather than a peer-reviewed benchmark, so comparisons must pin the upstream revision and invoke its actual algorithmic functions rather than compare prompt snippets alone.

Semantic-Motion adopts the overlapping-window principle but extends the data contract. Macro windows propose task boundaries and high-level intent; dense micro windows describe shorter action primitives within each accepted segment. For paired input, frames from fixed and ego views are extracted at identical source indices and interleaved by timestamp before inference. Segment records retain frame intervals, evidence by view, macro intent, micro-instructions, trajectory references and augmentation metadata. This richer output is needed for later synchronization and semantic checks.

2.5 Demonstration Data Quality

Demonstration scale does not remove the need for quality control. BridgeData V2 shows the value of task and environment diversity for scalable robot learning [10], and MimicGen demonstrates how object-centric subtask structure can expand a small set of demonstrations into a larger dataset [6]. Diffusion Policy further illustrates that modern visuomotor policies learn distributions over temporally structured actions [1]; corrupted timing or physically implausible trajectories are therefore not benign metadata defects.

For this project, quality has at least three dimensions. *Contract quality* concerns file availability,

paired-view synchronization, coordinate units and motion coverage. *Physical quality* concerns velocity, acceleration, jerk, directional oscillation and timestamp regularity. *Semantic quality* concerns whether the text identifies the observed actors, objects, task and temporal order without hallucination. A useful filter must keep these dimensions separate in diagnostics even if it ultimately emits a binary decision.

2.6 Engineering Gap

Existing systems tend to optimize one part of the chain. VLA models focus on policy learning; benchmark suites focus on standardized tasks; scalable data generation systems focus on obtaining more trajectories; and video-to-task tools focus on segmentation and instruction generation. The project materials identify a gap between these components: there is no lightweight, local, auditable path that starts with human or simulated motion, creates synchronized multi-view evidence, generates multi-granularity language, attaches real motion and returns explainable quality decisions.

Semantic-Motion addresses this gap through explicit file and schema contracts rather than a monolithic model. The resulting design can be tested stage by stage, can retain independent artifacts for audit, and can expose failure reasons to both scripts and a Web dashboard. Its remaining gap is equally important: prompt-constrained augmentation is not yet deterministically fact-checked, and not every recorded synchronization or confidence diagnostic currently changes the final decision. Chapters 3 and 4 therefore distinguish the target fail-closed architecture from the precise behavior of the present implementation.

Chapter 3

System Requirements and Overall Design

3.1 Inputs, Outputs and Use Cases

The system accepts three principal source forms:

1. Module D directories containing the required fixed-view video, ego-view video and sample-specific trajectory;
2. a versioned `ViewBundle` or dataset manifest referencing paired views and a trajectory; and
3. compatibility inputs consisting of a single video or a directory of pre-extracted frames, which are restricted to debugging unless paired-view requirements are explicitly relaxed.

For each manifest sample, the automated path writes three independent artifact pairs: a perception JSON, a machine-readable sample JSONL with a pretty JSON equivalent, and a refined JSONL with a pretty JSON equivalent. The refined record exposes the dimensions required by the dashboard: `motion_quality_score`, `semantic_label`, `semantic_confidence`, `decision`, `reason_codes` and nested diagnostics.

The main use cases are:

- **Batch dataset preparation**: process every sample in a Module D manifest while retaining independent outputs;
- **Targeted debugging**: execute one `target_sample_id` and inspect its intermediate artifacts;
- **Controlled view study**: run fixed, ego and fused perception modes for ablation;
- **Quality review**: sort or inspect samples by motion score, semantic label, decision and reason code;
- **Regression evaluation**: run deterministic contract tests, motion corruptions and, where gold labels exist, temporal and decision metrics; and
- **Operator monitoring**: launch the pipeline through the Web interface and poll progress without holding one synchronous HTTP request open.

3.2 Design Goals

The architecture is guided by six design goals.

Evidence alignment. Fixed video, ego video, motion and text must refer to one sample and one temporal interval.

Explicit contracts. Cross-module communication must use versioned, validated models or documented JSON/JSONL fields rather than implicit filename assumptions alone.

Independent auditability. Each processing stage must retain its own per-sample artifact so that a final decision can be traced backwards.

Conservative quality control. Missing physical or semantic evidence should be visible and should ultimately lead to fail-closed policy behavior. The current decision wiring is assessed against this target.

Replaceable components. Recognition models, instruction rewriters, semantic verifiers and motion

sources must be replaceable behind narrow interfaces.

Operational observability. Long-running external API calls must expose status, progress, current stage and captured logs.

3.3 Overall Architecture

The architecture is a staged transformation with a feedback interpretation rather than an online control loop. Module C reason codes can inform later changes to prompts, rendering and thresholds, but they do not automatically modify upstream models during one execution.

Table 3.1. End-to-end architecture and principal artifacts.

Stage	Primary implementation	Transformation	Principal output
Rendering (D)	`module_d/render_pipeline.py`	Imports FBX/BVH motion, constructs scene context, renders fixed/ego video and exports selected bone trajectories.	Two MP4 files and one trajectory JSON per action.
Indexing	`tools/prepare_module_d_manifest.py`	Validates expected files and creates portable sample references.	`data/module_d_output/manifest.json`.
Perception (A)	`src/semantic_motion/video_task.py`	Performs paired temporal sampling, VLM recognition, transition voting and macro/micro aggregation.	`VideoTaskRecord` v2 JSON.
Augmentation (B)	`src/semantic_motion/augmentation.py`	Produces source, template or LLM instruction variants with source-step and prompt provenance.	Augmented instructions embedded in task segments.
Preparation (C1)	`src/module_c/prepare_samples.py`	Converts task records to standardized samples and loads real motion tracks.	Per-sample JSONL and pretty JSON.
Refinement (C2)	`src/module_c/run_refinement.py`	Checks alignment, scores motion, verifies semantics and emits explanations.	Per-sample refined JSONL and pretty JSON.
Orchestration	`app.py`, `dashboard/src/App.jsx`	Runs selected stages asynchronously, reports progress and visualizes refined records.	Task-status API and Web dashboard.

The Python process remains the system of record. The React dashboard does not recompute scientific metrics; it normalizes scores for display and maps `keep`/`drop` to operator-facing labels.

3.4 Module Responsibilities and Boundaries

The practical Module A/B boundary is an in-process boundary: `VideoTaskPipeline` owns a `PerceptionStream` and an `AugmentationStream`, and `run_video_task.py` serializes their combined result. The Module C boundary is deliberately file-based so that perception records can be inspected or replaced before refinement.

Table 3.2. Module responsibilities and exclusions.

Module	Responsible for	Explicitly not responsible for
Rendering	Visual representation of raw motion, camera placement, simple scene context, video encoding and trajectory export.	Semantic labels, motion acceptance decisions or robot control.
Perception	Visual grounding, candidate boundaries, macro intent, micro-actions and evidence references.	Guaranteeing linguistic factuality or physical trajectory quality.
Augmentation	Instruction-style diversity, source-step linkage and rewrite provenance.	Changing observed actions or certifying final sample acceptance.
Refinement	Sample normalization, synchronization diagnostics, physical scoring, semantic verification, decisions and reason codes.	Repairing corrupted source motion or retraining upstream models.
Web layer	Task orchestration, polling, log exposure and visualization.	Changing model outputs, thresholds or scientific metrics.

3.5 Data Models and Interface Contracts

Pydantic models in `src/semantic_motion/models.py` define the upstream contracts, while dataclasses in `src/module_c/schema.py` define the refinement input and output. Every contract carries a stable `sample_id`; temporal segments add start/end frames and seconds; and provenance records the dataset, source, generator and Git revision. JSONL is used for streaming and machine processing, while pretty JSON is written as an equivalent inspection artifact.

Table 3.3. Critical cross-module fields.

Contract	Required core fields	Purpose
`ViewBundle`	`sample_id`, `timebase`, `views`, `trajectory`, `provenance`	Associates synchronized visual and physical evidence.
`TaskSegment`	frame/second interval, task text, macro intent, micro-instructions, confidence, evidence by view	Represents one temporally grounded semantic task.
`Sample`	`sample_id`, video/views, text, motion tracks, interval, provenance	Provides the normalized unit consumed by Module C.
`RefinementResult`	motion score, semantic label/confidence, decision, reason codes, auxiliary data	Supports acceptance, audit and visualization.

3.5.1 Paired Multi-view Video Contract

A formal sample contains at least the keys `fixed` and `ego`. Each maps to a `ViewStream` with a path, camera type, FPS, frame count and duration. `build_view_bundle` probes videos rather than trusting filenames, constructs a shared timebase from a reference stream, and records validation diagnostics for mismatched metadata. During fused perception, identical source-frame indices are extracted from each view and ordered fixed-then-ego for every timestamp. This ordering is stated in the VLM prompt so that transition indices refer to timestamps rather than individual images.

It is important to distinguish diagnosis from enforcement. At perception time, `validate_view_bundle` returns stable synchronization reasons, but `run_view_bundle` stores them with `validation_enforced=false`. Module C later probes the files again and adds missing-view, unreadable-view, FPS, frame-count and duration reason codes.

3.5.2 Shared Timebase and Motion Trajectory Contract

`SharedTimebase` contains FPS, frame count, duration in seconds and a `FrameMap` relating encoded frames to source frames. A `MotionReference` contains the trajectory path, source, spatial unit, coordinate frame and track names. The application-level orchestrator further requires the Module D convention `data/module_d_output/<sample_id>/<sample_id>_trajectory.json`; it rejects unexpected or escaping paths before starting model inference.

Module C converts supported Module D, LIBERO or standard JSON trajectories into named `MotionTrack` arrays containing positions and timestamps. Formal checks require increasing timestamps, supported units, real rather than dummy motion, and coverage of the sample interval. Track positions are normalized to metres before velocity, acceleration, jerk and jitter indicators are computed. The default configuration uses minimum aggregation across tracks, so the weakest configured track determines the aggregate motion score.

3.6 Observability and Visualisation Design

VLM and semantic-verifier calls can take substantially longer than an ordinary HTTP request. The FastAPI service therefore returns a UUID task identifier from `POST /run-pipeline` and executes the blocking command sequence as a background task. A lock-protected in-memory dictionary stores `queued`, `running`, `completed` or `failed` state, percentage progress, current step, selected sample, captured `stdout`/`stderr` and any error.

The React client polls `GET /get-status/{task_id}` every 1.5 seconds. It displays a progress bar, current stage and log count, then retrieves `GET /get-results` after completion. Results are read from independent `*_refined.pretty.json` files; the API combines records in memory for display but does not overwrite them as one evaluation artifact.

The dashboard presents processed count, keep rate, mean Motion Quality and Semantic Consistency rate. Motion scores are mapped from $[0, 1]$ to a 0–100 display scale and coloured red for 0–35, amber for 36–59 and green for 60–100. These colours are visual categories only and do not redefine the YAML threshold. The table retains the backend semantic label, confidence, decision and all reason codes.

This implementation is suitable for local single-process operation. Task state is lost when the server restarts and is not shared across workers. Durable deployment would require persistent task

storage, authentication, restricted CORS configuration and a dedicated job queue.

3.7 End-to-End Workflow

The automated formal path is:

1. Module D renders each source motion into fixed and ego videos and exports Root/Hand trajectories.
2. `prepare_module_d_manifest.py` validates each directory and writes one manifest entry per valid sample.
3. The orchestrator reads the manifest, optionally selects one `target_sample_id`, validates its trajectory convention and creates independent output paths.
4. `run_video_task.py` loads the `ViewBundle`. It creates overlapping 16-second macro windows with 8-second stride and 16 sampled timestamps by default.
5. The VLM proposes macro semantics and local transition indices. Hanning-weighted votes from overlapping windows are clustered into global cuts, subject to minimum segment duration.
6. Within each segment, dense 2-second micro windows with 1-second stride and four sampled timestamps produce ordered action primitives. The segment aggregates macro intent, micro-instructions, confidence, evidence and trajectory linkage.
7. Module B produces the configured number of source/template/LLM variants. Current LLM variants carry prompt and source-step provenance but are explicitly marked as not deterministically validated.
8. `prepare_samples.py` selects the configured segment- or video-level text, attaches the sample's real trajectory and writes independent JSONL/pretty JSON.
9. `run_refinement.py` checks cross-modal metadata, scores motion, invokes multi-view semantic verification and writes an independent refined record with explanations.
10. The task status is marked complete and the dashboard refreshes the available per-sample results.

The filename pattern is deliberately stable: `<sample_id>.json`, `<sample_id>_samples.jsonl` and `<sample_id>_refined.jsonl`, each with a corresponding pretty JSON where applicable. This preserves the semantic lineage from one manifest entry to one set of auditable downstream artifacts.

Chapter 4

Design and Implementation of the Four Modules

4.1 The Rendering Module

In the end-to-end data flow of the Semantic-Motion system, the Rendering Module serves as a critical bridge connecting raw motion data with Vision-Language Models (VLMs). Traditional Motion Capture data (such as FBX or BVH formats) essentially consists of a series of 3D coordinate point clouds and skeletal rotation matrices that change over time. Because general-purpose VLMs lack the capability to directly parse pure 3D geometric data, a significant ”representation gap” exists. The design objective of this module is to construct a fully automated, high-fidelity rendering pipeline that transforms abstract ”digital signals” into VLM-perceptible ”visual signals” (multi-view videos), while simultaneously exporting standardized 3D physical trajectories to provide high-quality data sources for downstream semantic perception and quality refinement. The core implementation of this module is built upon the Blender 5.1 Python API.

4.1.1 Motion Data Import and Scene Initialisation

To achieve zero-human-intervention processing for large-scale datasets, the rendering module first establishes a robust batch import and scene initialization mechanism. Upon pipeline initiation, the system iterates through the FBX or BVH motion files in the input directory, importing the armature and animation data via Blender's native interfaces. Addressing the issue of variable action lengths, the script automatically reads the `animation_data.action.frame_range` attribute of the armature to dynamically set the start and end frames of the current scene, thereby achieving adaptive animation durations.

In industrial-grade batch processing, memory leaks and scene pollution are common issues. To address this, the module implements a whitelist-based Trace-free Cleanup Mechanism. Before importing each new action, the system automatically traverses the current scene, destroying the temporary meshes and skeletal data generated in the previous iteration, while strictly preserving foundational scene objects such as lights, cameras, and the ground plane. This state isolation strategy avoids the overhead of repeatedly creating the base environment, significantly enhancing throughput. Furthermore, considering that external motion data may be corrupted, the import phase encapsulates a Try-Except fault-tolerance mechanism. When encountering fatal errors such as an ”invalid header,” the system automatically logs the error and skips the file, ensuring that the entire engineering pipeline does not crash due to a single corrupted data point.

4.1.2 Scene Context Generation

To assist the VLM in focusing on the action itself and accurately recognizing macro-intents, the module incorporates a dynamic Scene Context Generation algorithm based on semantic parsing and skele-

tal dynamic tracking. Pure actions (e.g., sitting down in mid-air) are prone to trigger VLM hallucinations or misjudgments. Therefore, by parsing keywords in the motion filenames and combining them with the skeletal coordinates in physical space, the system dynamically injects minimalist geometric anchors:

Vault/Step: When a vaulting or stepping semantic is identified, the system iterates through every frame of the animation to extract the highest point (peak of flight) of the Root/Hips bone along the Z-axis in the world coordinate system. Subsequently, the corresponding (X, Y) coordinates of that frame are projected onto the ground, and a rectangular cuboid obstacle is generated at this geometric center. This ensures that regardless of where the character initiates the jump, the obstacle perfectly aligns with its motion parabola.

Sit: The system shifts to tracking the lowest point of the Hips bone along the Z-axis and generates a cubic chair directly beneath it, resolving the clipping issue where the character appears to "sit in mid-air" during linear displacement.

Drop/Jump Down: For waterfall-type trajectories where the character drops downwards, the system reads the posture of the initial jump frame, iterates through bones such as the Ankle and Toe, and precisely locates the absolute lowest point of the sole. Simultaneously, the algorithm calculates the directional vector from the jump to the landing and applies a dynamic offset to the generated 2x2 meter platform in the opposite direction of the jump. This positions the character exactly at the edge of the platform in the initial frame, perfectly recreating the physical logic of "jumping down from a height."

4.1.3 Multi-view Video Synthesis

To meet the Semantic-Motion system's requirements for multi-granular semantic extraction, the rendering module deploys a dual-track virtual camera system within the 3D engine to simulate a real-world, integrated hardware-software capture environment:

Fixed View: Deployed globally within the scene, this view is utilized to capture the character's full-body kinematics, movement trajectories, and macro-spatial relationships with the environmental context.

Ego View: Aimed at simulating the robot's head-mounted perception. The script uses Python to traverse and lock onto key bones containing "Head" in their names, dynamically binding the camera to the character's head via the `parent_bone` attribute. Subsequently, the viewing direction is corrected using Euler Rotation, enabling it to track the character's line of sight with zero distance. This perspective is crucial for the downstream Perception Module to capture micro-action details, such as Hand-Object Interactions (HOI).

On the rendering output level, the system invokes the Blender EEVEE rendering engine to automatically overwrite the environment background with a solid color (mid-gray/black), eliminating ambient light noise. The output format is configured as a 1024×1024 resolution H.264 encoded MP4 video, ensuring that the VLM is provided with high-definition, interference-free visual inputs even under lower VRAM constraints.

4.1.4 3D Motion Trajectory Export

While generating visual signals, the rendering module is also tasked with extracting physical Ground Truths. Pure video data cannot directly provide precise physical metrics (e.g., jump height, accelera-

tion), which are indispensable for validating motion quality.

This section embeds a trajectory export algorithm within the rendering loop. The algorithm forces an update of the view layer via `bpy.context.view_layer.update()` and reads the product of specific Pose Bones and the `matrix_world` frame-by-frame to obtain absolute physical coordinates. To ensure compatibility with heterogeneous motion capture standards (e.g., Mixamo, CMU), the system features built-in skeletal mapping dictionaries that automatically match and extract the spatial coordinates of key nodes like the Root (center of mass), Hand_R, and Hand_L. The data is truncated to four decimal places to reduce storage redundancy and serialized into JSON format. This design bifurcates the pipeline: the rendered videos flow into the Perception Module, while the precise 3D trajectory data flows directly into the downstream Refinement Module, providing numerical bases for the final Motion Quality Scoring.

4.1.5 Rendering Outputs and Manifest

Following the automated processing of the aforementioned engineering pipeline, the Rendering Module stably generates three core deliverables in the output directory for every inputted raw motion file (e.g., `drop_from_roof.fbx`):

- **Fixed_View.mp4**: A third-person global video featuring the scene context.
- **Ego_View.mp4**: A first-person detail video closely following the character's head movements.
- `<action_name>_trajectory.json`: A 3D motion trajectory file containing the frame-by-frame (X, Y, Z) coordinates of core bones.

These standardized outputs completely eliminate the representation barriers of the raw data, laying a solid underlying data foundation for the system's subsequent semantic generation and multimodal data refinement.

4.2 The Perception Module

4.2.1 Paired-view Input and Synchronous Frame Sampling

4.2.2 Overlapping Macro Windows and Boundary Voting

4.2.3 VLM-based Structured Scene Understanding

4.2.4 Macro-Intent Recognition

4.2.5 Dense Micro-Instruction Parsing

4.2.6 Body, Contact, Posture and Trajectory Evidence

4.2.7 Temporal Task Boundary Detection

4.2.8 Segment-level Temporal Aggregation

4.2.9 Static Perception Benchmark

4.3 The Fact-preserving LLM Augmentation Module

4.3.1 Input and Interface

4.3.2 Instruction Reconstruction

4.3.3 Augmentation Strategies

4.3.4 Deterministic Fact Preservation and Hallucination Control

4.3.5 Output and Provenance

4.4 The Refinement Module

The Refinement Module is the final quality-control layer in the Semantic-Motion pipeline. The front modules render simulation data into videos, infer structured task text based on videos and generate augmented text. However, the generated description may contain an incorrect object, action, temporal order or target state. Meanwhile, the motion trajectory may contain spikes, jitter, invalid timestamps or insufficient observation. These samples are not suitable for downstream VLA training. The role of the refinement module is to ensure that only reliable samples are kept for downstream learning.

The refinement module filters samples containing video, motion and text through two perspectives, namely numerical analysis of motion quality and VLM-based verification of semantic consistency. The outputs of two channels are combined into an explainable keep or drop decision.

4.4.1 Sample Construction

The module first converts the task texts with corresponding videos produced the Perception and Augmentation Modules into a common JSONL representation. Each sample contains a unique sample id, a collection of one or more video paths indexed by views, the texts to be verified, and optional motion tracks.

This intermediate representation allows the refinement process independent from the internal structure of the earlier modules and enables the same procedure to operate on demonstrations of single view, rendering videos of multiple views, and externally prepared datasets.

Two sample granularities are supported. At segment level, every detected task segment becomes an independent sample. Its identifier combines the source demonstration name and segment identifier, and its motion data are restricted to the segment's start and end times. All synchronized views associated with that demonstration remain attached to the same sample. This mode is appropriate when deciding keep or drop individual sub actions. At video level, the complete video contents are included in the sample regardless of whether the demonstration has one view or multiple views. This mode is more suitable for checking whether a description correctly summarizes the overall tasks.

The text can be selected from the task instruction or augmented instructions. Empty texts, records without any valid video path, and records without task texts are skipped. The absence of one optional view does not make a sample invalid, as long as at least one usable view remains. If no motion track is contained in the sample, the refinement module can operate in semantic-only mode, determining to keep or drop the sample based solely on the task text and the corresponding video. Samples are written both in line-delimited JSON for machine processing, and optionally in indented JSON for manual inspection. This separation between processing and inspection formats improves batch efficiency and

ensures observability.

4.4.2 Motion Data Normalisation

Motion data from heterogeneous systems must be converted to consistent formats before a common quality metric can be applied. The refinement module represents motion as a dictionary of named tracks. Each position is converted to a three-dimensional float vector. A valid track must contain at least four observations, have the same number of positions and timestamps, and use strictly increasing timestamps. Invalid tracks are dropped during preparation.

Four input formats are currently supported. Firstly, LIBERO trajectories provide the position of agent's end effector in each action step. These positions are converted into a continuous end-effector track. If the source includes an FPS value, timestamps are derived from the step time and FPS. Secondly, trajectories generated by the rendering module are composed by the three-dimensional location of each selected body joint in every animation frame. The refinement module sorts the frames in chronological order and converts their frame numbers into times, then constructs separate trajectories for the body root, right hand and left hand. Thirdly, a standard single-trajectory file directly provides an ordered list of three-dimensional positions together with the corresponding observation times. The module treats this list as an unnamed trajectory. Finally, a standard multi-trajectory file provides several named position sequences and their observation times. These trajectories and their original names are retained.

For segment-level samples, every track is temporally cropped to the corresponding segment interval and evaluated again. The loader also supports a plug-in interface for additional motion formats. If no real trajectory can be resolved, the pipeline can generate a short linear placeholder trajectory to keep the engineering workflow executable. This placeholder will not be interpreted as evidence of genuine motion quality. By default, the refinement module only verifies semantic analysis when there is no motion track in the sample.

4.4.3 Motion Quality Scoring

For a valid track with positions $\mathbf{p}_i$ at times t_i, let $\Delta t_i = t_{i+1} - t_i$. The implementation clips each time interval from below by $\varepsilon = 10^{-6}$ for numerical safety and computes discrete velocity, acceleration, and jerk as

$$\mathbf{v}_i = \frac{\mathbf{p}_{i+1} - \mathbf{p}_i}{\max(\Delta t_i, \varepsilon)}, \qquad \mathbf{a}_i = \frac{\mathbf{v}_{i+1} - \mathbf{v}_i}{\max(\Delta t_{i+1}, \varepsilon)}, \qquad \mathbf{j}_i = \frac{\mathbf{a}_{i+1} - \mathbf{a}_i}{\max(\Delta t_{i+2}, \varepsilon)}.$$

For uniformly sampled motion, all intervals equal the constant sampling period Δt, so the three denominators above have the same numerical value. The velocity, acceleration, and jerk abnormality ratios are the proportions of their Euclidean magnitudes that exceed the configured limits:

$$r_v = \frac{1}{N_v} \sum_i \mathbb{I}(\|\mathbf{v}_i\|_2 > v_{\max}), \quad r_a = \frac{1}{N_a} \sum_i \mathbb{I}(\|\mathbf{a}_i\|_2 > a_{\max}), \quad r_j = \frac{1}{N_j} \sum_i \mathbb{I}(\|\mathbf{j}_i\|_2 > j_{\max}).$$

Here, $\mathbb{I}(\cdot)$ is the indicator function. The default limits are $v_{\max} = 2.5$, $a_{\max} = 6.0$, and $j_{\max} = 20.0$. They represent the largest motion magnitudes treated as normal for an individual sample. Exceeding a threshold marks that sample as abnormal, but does not by itself reject the entire track. The limits

are configurable and should be adjusted when the expected motion scale or sampling characteristics of a dataset differ.

Jitter is measured from changes in the full three-dimensional velocity direction. For every velocity satisfying $\|\mathbf{v}_i\|_2 > 10^{-8}$, define

$$\mathbf{u}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2},$$

and let

$$\mathcal{D} = \left\{ i : \|\mathbf{v}_i\|_2 > 10^{-8} \ \wedge \ \|\mathbf{v}_{i+1}\|_2 > 10^{-8} \right\}.$$

The jitter ratio is the proportion of valid adjacent velocity pairs whose directions differ by more than 90°:

$$r_d = \begin{cases} \dfrac{1}{|\mathcal{D}|} \displaystyle\sum_{i \in \mathcal{D}} \mathbb{I}\left(\mathbf{u}_i^\top \mathbf{u}_{i+1} < 0\right), & |\mathcal{D}| > 0, \\ 0, & |\mathcal{D}| = 0. \end{cases}$$

Normalization $\mathbf{v}_i$ removes the influence of magnitude, so this measure responds only to changes in direction. Velocities below 10^{-8} are excluded because their directions are undefined or numerically unstable. Since both $\mathbf{u}_i$ and $\mathbf{u}_{i+1}$ are unit vectors, their dot product equals the cosine of the angle θ_i between them:

$$\mathbf{u}_i^\top \mathbf{u}_{i+1} = \cos\theta_i.$$

A negative dot product therefore indicates $\theta_i > 90°$, which the implementation counts as a rapid direction reversal. Consequently, for $r_d \in [0, 1]$, a value near zero indicates few such reversals, while a larger value indicates more frequent directional oscillations. If there is no valid adjacent velocity pair, r_d is defined as zero.

Two additional measures detect timing anomalies. The interval coefficient of variation c_t and the largest-gap ratio g_t are

$$c_t = \frac{\mathrm{std}(\Delta t)}{\max(\mathrm{mean}(\Delta t), \varepsilon)}, \qquad g_t = \frac{\max_i \Delta t_i}{\max(\mathrm{median}(\Delta t), \varepsilon)}.$$

The coefficient c_t measures the overall relative variation of the sampling intervals. It is zero when all intervals are equal and increases as the sampling schedule becomes less regular. The ratio g_t compares the largest interval with the median, which represents the typical frame interval and is less sensitive to isolated gaps than the mean. Thus, $g_t = 1$ for uniform sampling, while a significantly larger value indicates an unusually long interval that may result from a dropped frame or a timestamp shift. The terms ε only prevent division by a reference interval near zero.

The implementation therefore forms six normalized penalties. The first three use a ratio of abnormal samples of 0.2; the other three use their configured limits $r_{d,\max} = 0.30$, $c_{t,\max} = 0.25$, and $g_{t,\max} = 2.5$:

$$q_v = \min(1, r_v/0.2), \qquad q_a = \min(1, r_a/0.2), \qquad q_j = \min(1, r_j/0.2), \qquad q_d = \min(1, r_d/r_{d,\max}),$$

$$q_c = \min(1, c_t / c_{t,\max}), \qquad q_g = \min\left(1, \frac{\max(0, g_t - 1)}{\max(g_{t,\max} - 1, \varepsilon)}\right).$$

The quality score for one track is

$$s_{\text{track}} = \text{clip}\left(1 - \frac{q_v + q_a + q_j + q_d + q_c + q_g}{6}, 0, 1\right).$$

Consequently, a smooth trajectory with regular sampling receives a score near one, whereas frequent kinematic spikes, direction reversals, irregular intervals, or large time gaps reduce the score. The corresponding reason codes are `high_velocity_spikes`, `high_acceleration_spikes`, `high_jerk_spikes`, `high_jitter`, `irregular_frame_intervals`, and `drop_frame_or_time_shift`. Tracks with fewer than four samples, non-finite values, or non-increasing timestamps receive a score of zero and a corresponding validation reason.

When multiple tracks are present, their scores are aggregated using either the minimum or the mean. The default is the minimum, so a single track with a poor rating is sufficient to drag down the complete sample's motion score. This is appropriate for strict data cleaning because a smooth root trajectory should not conceal an unstable hand trajectory. Mean aggregation is available when overall motion quality is more important than the weakest component.

4.4.4 Semantic Consistency Verification

Motion smoothness alone cannot determine whether a text describes the visual task correctly. The semantic channel submits all available views, together with the candidate description, to the multimodal verifier. A LIBERO sample is verified from its video of single agent view and eye in agent's hand. A rendering module sample is verified jointly from the egocentric and fixed third-person views. The first-person view provides evidence of subject interaction, while the fixed view provides motions of full body and scene context. By treating these two videos as complementary observations of the same sample, the system can avoid making conflicting decisions for each camera.

The verifier uses qwen3-vl-plus through an endpoint compatible for OpenAI. For each available view, a local video of no more than 18 MB can be encoded and sent directly. For a larger video, the system uniformly extracts eight temporally ordered JPEG frames at quality 85. The view name is retained when constructing the request so that evidence from the egocentric and fixed cameras can be distinguished.

The verification prompt evaluates seven dimensions: actor correctness, object correctness, action correctness, temporal correctness, goal and final-state correctness, hallucination, and evidence sufficiency. The verifier makes decisions relying only on visible evidence. Ambiguous evidence produces an "uncertain" result, rather than an unsupported guess.

The normalized output contains a label, which may be "consistent", "uncertain" or "inconsistent", a confidence value between zero and one, a list of error types, and a suggested replacement text. Confidence represents confidence in the assigned label. For example, "inconsistent" with high confidence indicates strong evidence of a mismatch. Error tags identify failures such as object_mismatch, action_mismatch, possible_hallucination, and insufficient_evidence.

4.4.5 Deterministic Cross-modal Alignment Checks

The final decision combines two quality channels conservatively. A sample is retained only when its semantic label is "consistent" and the motion score meets or exceeds the default threshold of 0.35. If no motion score is available, semantic consistency alone determines whether the sample can be retained.

A sample with correct semantical descriptions is dropped when its motion score is below the threshold. A smooth trajectory is also dropped when its descriptions are uncertain or inconsistent. When motion is absent, the sample enters semantic-only mode and may still be kept if its semantic label is "consistent". This procedure ensures that the entire process is operational while the condition that motion was not evaluated is explicitly recorded.

4.4.6 Fail-closed Refinement Decision and Explainable Outputs

Each result of refinement module records the sample identifier, motion-quality score, semantic label, semantic confidence, final decision, and a list of reason codes. It also retains the scores and abnormality ratios of per track, semantic error types, suggested text, and original candidate text. Results are saved as JSONL for downstream processing and as formatted JSONL for manual review. This output design makes rejection traceable. A drop decision may be attributed to a threshold failure such as "low_motion_score", to an issue of a specific track such as "Hand_R: high_jitter", or to a semantic problem such as "object_mismatch".

The Refinement Module closes the engineering loop of four modules. The Rendering Module supplies visual and motion signals. The Perception Module extracts structured semantics. The Augmentation Module generates text variants. The Refinement Module determines whether the aligned sample is sufficiently reliable.

Chapter 5

System Integration, Demonstration and Evaluation

This chapter evaluates Semantic-Motion Engine as an integrated engineering system, rather than just four isolated models. The four stages follow the actual data flow. First, the Rending Module converts motion or simulation data into videos of first-person and third-person view. Second, the Perception Module extracts structured task semantics from related videos. Third, the Augmentation Module generates command variants. Fourthly, the Refinement Module evaluates the motion quality and semantic consistency of samples to determine whether to filter of retain them. The evaluation verifies the operation of the individual modules and the transfer of videos, motions and text between consecutive stages.

5.1 Integration Environment and Technology Stack

The integrated software pipeline was implemented in Python 3.10 or later. OpenCV is used for video decoding, frame sampling, and local video preprocessing. Structured data are defined as Python classes with fields and data types, and are exchanged between modules via JSON or JSONL files. YAML configuration files store motion thresholds, synchronization requirements, settings of semantic verifier, and model parameters.

The Rendering Module uses Blender to convert motion files into videos of egocentric and fixed view while exporting three-dimensional joint trajectories. Visual-language inference is implemented through a DashScope API compatible with OpenAI. API credentials, endpoint addresses, and model identifiers are supplied through environment variables rather than source files. This avoids embedding secrets in the repository and allows to change inference models without modifying project codes.

5.2 Module Integration and Interface Verification

The principle integration object is a view bundle. It associates a sample identifier with a shared time base, fixed and egocentric videos, a trajectory source, and source information. The views in a valid formal multi-view sample share the same frame rate, frame count, and duration, allowing observations from different cameras to be compared at corresponding times.

The Perception Module processes the view bundle and produces a versioned video-task-record. This record contains corresponding video path, sampled multi-view frames, temporal task segments, macro-intents, micro-instructions, detected objects and spatial relations. The Augmentation Module receives the structured segment description, and every generated variant retains its relationship between source steps.

Before refinement, the perception record is converted into a uniform sample, containing candidate texts, video views, segment boundaries and motion tracks. LIBERO's positions of end effectors and Rendering Module's trajectories of body joints are normalized into the track representation with the

same name. This interface allows the motion scorer to operate without knowing the original dataset format.

Interface verification was performed on the LIBERO sample "put_the_bowl_on_the_plate", which contains views of agent and eye in hand. Both views contain 90 frames at 20 frames per second and have a duration of 4.5 seconds. The synchronization check reports a valid sample of paired views. The first generated segment covers 0.00–3.35 seconds, and its end-effector trajectory covers the same interval. The second segment covers the remainder of the demonstration. The provenance records the dataset, source identifier, generator, and source revision. These checks show that visual, textual, temporal, and motion data remain associated after passing through multiple modules.

Schema validation prevents several engineering failures. For example, a view cannot be detached from its trajectory without detection and motions in segment level cannot be evaluated against unrelated time intervals. Intermediate files in standard format and structured error codes also prove for observability at module boundaries.

5.3 End-to-End Demonstration

The main end-to-end demonstration uses the Module D jump_down motion. The Rendering Module imports the action of motion capture into Blender, identifies it as a downward jump action, and places a platform relative to the character's take-off position and movement direction. It renders Fixed_View.mp4 from the scene camera and Ego_View.mp4 from a camera attached to the head bone. At the same time, it exports jump_down_trajectory.json, which contains the world coordinate positions of Root, Hand_R and Hand_L at each animation frame. The resulting paired videos each contain 70 frames at 30 frames per second and last 2.33 seconds.

The rendered files are indexed by manifest.json as one jump_down view bundle. The Perception Module processes the fixed and egocentric videos jointly in the mode of fused view and associates the exported joint trajectory to the same record. It identifies a cut at frame 35 and produces two temporal segments. The first segment, from 0.00 to 1.17 seconds, is described as the humanoid crouching on a block, jumping upward, reaching with both hands, and catching a floating low-poly head-like object. The second segment, from 1.17 to 2.33 seconds, is described as a transition from standing to a low crouched posture supported by the hands and feet without interaction with other objects. The stored perception record was generated with qwen3-vl-plus.

The Augmentation Module generates three rewritten instructions for each segment, producing six variants in total. These variants retain the associations with the four source microsteps in their respective segments and take strategies such as temporal compression, goal-oriented framing, stage-based decomposition and spatial context enhancement.

Before refinement, prepare_samples combines the two segment descriptions into one candidate in video level while retaining both source segment identifiers, video paths 30 FPS shared time base, and all three motion tracks. The synchronization check confirms that the fixed and egocentric videos both contain 70 frames and have matching 2.33 second durations. The Root, Hand_R and Hand_L tracks cover 0.00–2.30 seconds. The visual and motion evidence are temporally aligned.

The Refinement Module evaluates the three tracks with the aggregation of minimum score. Root and Hand_L each score approximately 0.5000, while Hand_R scores 0.4918. Therefore the aggregate motion-quality score is 0.4918. All three tracks trigger high-velocity, high-acceleration, and high-jerk diagnostic codes, reflecting the rapid take-off and landing dynamics of the jump. Since the motion

threshold is 0.35, the aggregate score under the default configuration still passes the motion gate. Joint multi-view semantic verification labels the combined task text "consistent" with confidence 0.95 and returns no semantic error types.

The recorded refinement decision is "keep". This decision follows the Refinement Module rule: a sample is retained only when the motion-quality score is not below the configured minimum and the semantic label is "consistent". Otherwise it is dropped. In the present case, both conditions are satisfied, so the sample is kept even though the reason code contains spike diagnostic information of track.

5.4 Perception Module Evaluation

The static perception module benchmark was evaluated on 30 scenarios of robot tasks and structured ground truth. It evaluates object recognition, task classification, action sequence similarity, semantic similarity, and spatial reasoning.

Six complementary metrics are used. Object recognition F1 measures set overlap between predicted and ground-truth object names after normalization. Task classification accuracy is a binary score that equals one only when the predicted task type matches the labelled type exactly. ROUGE-L of action sequence compares the longest common subsequence between the predicted and reference action plans and therefore rewards both content and order. Semantic similarity aggregates the predicted objects, task type, actions, and target into a single bag-of-words representation and computes cosine similarity against the corresponding ground-truth text. Spatial reasoning accuracy counts the fraction of labelled spatial relations recovered in the prediction through substring matching. The composite score is the equally weighted average of these five metrics, each contributing a weight of 0.2. The results are shown below.

Metric	Score
Object recognition F1	0.7602
Task classification accuracy	1.0000
Action-sequence ROUGE-L	0.7176
Semantic similarity	0.9273
Spatial reasoning accuracy	0.6278
Composite score	0.8066

Table 5.1. Static Perception Module benchmark results on 30 scenarios using Qwen-VL-Plus.

The model correctly classified the task type in all 30 synthetic scenes and achieved high overall semantic similarity. This indicates that the structured output format can represent broad task intent effectively. Object recognition and action-sequence matching were weaker but remained substantially above zero. Spatial reasoning produced the lowest score. Inspection of predictions shows that support surfaces and appliance subcomponents were often described too generically, causing mismatches even when the overall task was understood.

5.5 Pinned Video2Tasks Full-pipeline Comparison

5.6 Refinement and Alignment Evaluation

5.6.1 Human Gold Annotation Protocol

5.6.2 Temporal Boundary and Segment Metrics

5.6.3 Keep/Drop and Confidence Calibration

5.6.4 Controlled Motion Corruptions

5.7 Cross-module Case Studies

5.8 Component and Configuration Analysis

5.9 Engineering Acceptance Summary

Chapter 6

Engineering Discussion

6.1 Interpretation of System Results

From the perspective of end-to-end execution, the system demonstrates exceptional capabilities in ”macro-task understanding,” yet reveals clear performance bottlenecks in ”micro-action and spatial parsing.” The Vision-Language Model (VLM) achieves a high accuracy rate in recognizing macro-intents (e.g., ”moving a box”), indicating that the large model's prior knowledge of general scenes generalizes well to the robotic rendering perspective. However, when processing spatial relationships and fine-grained operations (e.g., determining the exact distance and contact state between the end-effector and the object), the accuracy drops significantly.

Furthermore, the system outputs reveal a strong presence of cascading errors. A perceptual error or hallucination by the VLM in a single frame directly propagates into the temporal boundary detection of the task, leading to over-segmentation or erroneous merging of action segments. This explains why relying solely on vision-based large model predictions is insufficient, and it validates the absolute necessity of the dual-channel refinement mechanism (incorporating both ”motion trajectory quality” and ”semantic consistency”) in the Refinement Module. The dual-channel approach is significantly more effective at intercepting misleading data samples than relying on a single visual metric.

6.2 Engineering Strengths

The greatest engineering value of this project lies in the construction of a runnable, observable, and extensible closed-loop data infrastructure, rather than merely algorithmic fine-tuning:

Lightweight and Low Resource Dependency: The pipeline design drastically lowers the computational threshold. By calling cloud-based VLM APIs for visual understanding tasks, the system can run smoothly without the need for local high-performance GPU clusters.

High Modularity and Multi-source Compatibility: The four modules of the system are highly decoupled. This not only allows for seamless integration of more advanced VLMs or LLM rewriters in the future, but also enables the system to be compatible with highly divergent data sources. Whether processing raw FBX motion capture data with complex skeletal hierarchies or standardized LIBERO robotic trajectories, the system can clean and align them through unified data interfaces.

Strong Interpretability of Decisions: The automated data refinement process is not a black box. The output from the Refinement Module is not just a binary keep/drop decision; it includes detailed motion anomaly penalties (e.g., `high_jerk_spikes`) and semantic mismatch reason codes, providing solid data support for subsequent manual review and system debugging.

6.3 Failure Mode Analysis

During actual engineering execution, the system exposed several typical failure modes:

Visual Feature Confusion and Object Misidentification: The VLM often misidentifies interfer-

ing objects in the background as target objects, or experiences semantic confusion between visually similar components.

Information Loss due to Temporal Sampling: The uniform frame sampling strategy, adopted to balance cost and efficiency, may inadvertently miss critical interaction moments that last only for a fraction of a second, resulting in gaps in the generated micro-instruction descriptions.

Spatial Alignment and Rendering Biases: In the Rendering Module, when simple scene contexts (like obstacles or boxes in a vaulting action) are dynamically generated based on skeletal trajectories, spatial misalignments occasionally occur between the generated obstacle and the actual skeletal movement. This spatial dislocation in the rendered environment directly induces the VLM to produce false ”non-contact” or ”collision” hallucinations.

Boundary Collapse Triggered by Single-frame Hallucinations: Even if the action itself is continuous, if the VLM hallucinates in a single frame (e.g., falsely concluding that the target object has changed), the temporal segmentation algorithm will misjudge it as the start of a new task, leading to a logical fragmentation of the entire task sequence.

6.4 Limitations

Although Semantic-Motion provides a complete end-to-end pipeline, the current system is still constrained by several key factors:

Embodiment Gap: The Rendering Module currently processes human skeletal motion capture data. The degrees of freedom and kinematic characteristics of human motion possess an inherent embodiment gap compared to actual robotic arms (like UR5 or Franka). This implies that high-quality action descriptions refined based on human models may still require further alignment mapping before being directly transferred to robotic action generation models.

Lack of Large-scale Gold-label Data: The evaluation of Module C is currently limited by a small scale of video cases and static benchmarks (predominantly synthetic images). The lack of large-scale, human-expert annotated datasets for real robotic operations makes it difficult to rigorously prove the absolute performance gain of the dual-channel refinement on the final VLA training efficacy from a statistical standpoint.

Uncontrollability of External Dependencies: While heavily relying on cloud APIs brings convenience, it introduces risks regarding reproducibility. Silent updates to the underlying cloud models may result in different semantic outputs when feeding the exact same video input.

Limitations of Text Augmentation: Although the current rule-template-based Augmentation Module guarantees zero-hallucination, it sacrifices the rich diversity of natural language. It cannot generate instruction variants encompassing complex environmental modifiers or tonal shifts like a native LLM.

6.5 Engineering Trade-offs

The core process of building this system is essentially a collection of engineering trade-offs:

Convenience of Cloud APIs vs. Controllability of Local Deployment: Utilizing cloud APIs lowered the initial construction cost and hardware threshold but sacrificed local execution determinism and version-locked reproducibility.

Cost Advantage of Uniform Sampling vs. Temporal Information of Keyframes: Frame-by-

frame analysis carries an exorbitant computational overhead, making uniform sampling a necessary compromise. While it improves batch processing efficiency, it inevitably increases the risk of missing high-frequency motion details.

Safety of Template Augmentation vs. Diversity of LLMs: Considering the strict accuracy requirements for VLA instructions, we opted for a template-based rewriting strategy, sacrificing linguistic diversity in exchange for 100% semantic consistency.

Strict Aggregation (min) vs. Lenient Filtering (mean): When processing multi-trajectory motion scoring, using min aggregation maximizes the interception of anomalous trajectories (high precision), but it is also prone to incorrectly discarding high-value samples that contain minor, localized noise (low recall). This requires dynamic adjustment based on dataset scale during the actual construction of training sets.

6.6 Evaluation Validity

The validity of the current system evaluation methods can be examined across the following four dimensions:

Internal Validity: Through rigorous component ablation studies, we effectively validated the logical superiority of "dual-channel refinement" over "single-metric refinement."

External Validity: The current static benchmarks focus on synthetic images and five specific categories of robotic tasks. The system's generalization capabilities in handling extreme lighting conditions or open-vocabulary long-tail real-world scenarios require further empirical validation.

Construct Validity: Utilizing the ratio of jitter and jerk in motion trajectories as metrics for action data quality is theoretically sound in physics and control theory, effectively reflecting the smoothness and usability of the demonstration data.

Reproducibility: Although the code architecture is fully open-source and data formats are standardized, achieving completely consistent reproducibility across different time periods remains inherently challenging due to the reliance on external large model APIs.

6.7 Implications for VLA Data Pipelines

The engineering practice of Semantic-Motion offers several core insights for data engineering of future large-scale VLA models:

First, independent vision-language understanding validation is mandatory before feeding data into action generation models. Raw collected motion data contains significant amounts of unintentional wandering and trial-and-error; all trajectories cannot be blindly accepted.

Second, automatically generated task labels must never be used directly (i.e., "nakedly"). Pseudo-labels generated by large models must undergo a controlled feedback loop for rigorous review.

Finally, data refinement must move towards multimodal joint verification. Future filtering systems must evaluate not only whether the textual description matches the video (semantic dimension) but also whether the trajectory is physically smooth (physical dimension). Only samples where visual evidence and motion evidence corroborate each other can support the generalized learning of next-generation, high-performance robots.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

Report only automatically verified or independently evaluated results. Mark API, Blender, and pending-human evidence explicitly; do not infer completion.

7.2 Future Work

Bibliography

[1] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. doi: 10.15607/RSS.2023.XIX.026. URL https://www.roboticsproceedings.org/rss19/p026.html.

[2] Danny Driess, Fei Xia, Mehdi Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488, 2023. URL https://proceedings.mlr.press/v202/driess23a.html.

[3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of the 8th Conference on Robot Learning*, volume 270, pages 2679–2713, 2024. URL https://mlanthology.org/corl/2024/kim2024corl-openvla/.

[4] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract.html.

[5] Hao Luo, Ye Wang, Wanpeng Zhang, Sipeng Zheng, Ziheng Xi, Chaoyi Xu, Haiweng Xu, Haoqi Yuan, Chi Zhang, Yiqing Wang, Yicheng Feng, and Zongqing Lu. Being-H0.5: Scaling human-centric robot learning for cross-embodiment generalization. *arXiv preprint arXiv:2601.12993*, 2026. doi: 10.48550/arXiv.2601.12993. URL https://arxiv.org/abs/2601.12993.

[6] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1820–1864, 2023. URL https://proceedings.mlr.press/v229/mandlekar23a.html.

[7] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of household tasks for generalist robots. In *Proceedings of Robotics: Science and Systems*, 2024. doi: 10.15607/RSS.2024.XX.050. URL https://roboticsproceedings.org/rss20/p050.html.

[8] Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation*, pages 6892–6903, 2024. URL https://arxiv.org/abs/2310.08864.

[9] Video2Tasks Contributors. Video2Tasks: Split multi-task robot videos into single-task segments with auto-generated instructions. https://github.com/ly-geming/video2tasks. Open-source software repository, accessed 15 July 2026.

[10] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1723–1736, 2023. URL https://mlanthology.org/corl/2023/walke2023corl-bridgedata/.

[11] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag Sanketi, Grecia Salazar, Michael Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

Chapter A

VLM System Prompts

Chapter B

Semantic Consistency Prompt

Chapter C

Data Schemas

Chapter D

Configuration

Chapter E

Additional Results

Chapter F

User Guide